Knowledge Distillation and Deployment of a Command-Parsing Language Model on a Local Cloud

CS3081 Cloud Computing, Mini Project | Akash Mishra | Roll no. 230101101
Repository: https://github.com/akash2101h/cloud-computing-assignment

Conventions. Numbers in tables are **measured** on my laptop (WSL2, 8 CPUs, 10 GB RAM, single-node k3d) and the raw evidence is in `results/`. Anything that is an assumption is listed in the final section and marked (A).

1. Architecture overview

A k3d cluster (`cloudlab`, config in `cluster/k3d.yaml`) holds three namespaces: **data** (SeaweedFS S3 on a PersistentVolumeClaim), **pipeline** (two teacher-labelling Jobs) and **app** (FastAPI Deployment, Service, Traefik Ingress, HPA). Ollama runs phi3.5 on the host and is reached by the Jobs over the host network. The student (Pythia-70M) is trained on the host, stored in the S3 bucket `models` and in GitHub Releases, and downloaded by each Pod at start-up. Credentials live only in Kubernetes Secrets created at deploy time; none are in Git (gitleaks reports no leaks).

2. Task 1: object storage and parallel teacher labelling

Setup. SeaweedFS runs in `data` with buckets `datasets` and `models`. The pool was uploaded to `datasets`. The teacher is phi3.5 via Ollama with temperature 0 and seed 42. The prompt (`prompts/teacher_prompt.txt`) lists the four keys, eight intents, the three slot rules and 14 hand-written examples, and ends with a `JSON:` cue. Stop sequences (`\n\n`, `Command:`, `Note:`) end generation after one JSON line. My first prompt gave only 60% valid JSON on 20 validation commands because the model kept inventing new commands; the revised prompt gave 100%. The prompt was then locked before the 3,000-command run.

Sharding. Worker *k* of *N* takes the commands whose 0-based line number n satisfies n mod N = k. I ran N = 2, so shard 0 got the even lines and shard 1 the odd lines, each as its own Kubernetes Job in `pipeline`. The rule is deterministic, needs no coordination between workers, and cannot produce overlaps or gaps. Each worker uploads its output every 20 records and resumes from its own object, so a crash loses little work. Both workers share one CPU and one Ollama server, so two workers do not double throughput; they exist to show the pattern.

Check	Measured result
Records per shard	shard-0: 1500, shard-1: 1500
Coverage of ids	3000 total, 3000 unique, 0 missing, 0 repeated (results/shard_check.txt)
clean.py reproducibility	two runs gave the same SHA-256 a87c16a9...5132, equal to train.sha256

Cleaning counts (rules applied in the order a, b, c, d; a record counts under the first rule it fails; duplicates removed last, keeping the smallest id):

Input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
3000	4	1	10	104	50	2831

Check: 3000 = 4 + 1 + 10 + 104 + 50 + 2831. Training used the first 1,500 of the 2,831 kept records, in id order.

3. Task 2: training the student (sequence-level distillation)

Setup. `make train` reads only `training/config.yaml`: seed 42, learning rate 1e-4, batch size 8, 5 epochs, max length 128, linear schedule with 10% warm-up, AdamW, 8 threads; Python 3.11.18, torch 2.15.0, transformers 5.22.2. Each example is `Command: ...\nJSON: ...` plus the end-of-sequence token, and the loss is computed on the JSON part only. `make train SUBSET=50` ran to the end. v2 uses 6 epochs (`training/config_v2.yaml`).

Epoch	1	2	3	4	5	6
v1 loss	0.3763	0.1563	0.0808	0.0409	0.0159	-
v2 loss	0.3865	0.1606	0.1037	0.0583	0.0314	0.0103

Artefacts. Models are under `models/v1/` and `models/v2/` in SeaweedFS (weights, tokenizer, version_info.json, archive, model.sha256). The same archives are GitHub Release assets. v1 SHA-256: fa6a0b15...be72; I downloaded the Release asset and its hash is identical to `models/v1/model.sha256`. v2: 7cf0bc91...f0cd.

Evaluation on the 100 labelled validation commands (percent; greedy decoding for the models):

Metric	Teacher phi3.5	Untrained Pythia-70M	Student v1	Student v2
json_validity	98	0	100	99
intent_accuracy	87	0	86	85
date_accuracy	82	0	85	81
time_accuracy	84	0	86	87
place_accuracy	92	0	94	91
full_frame	57	0	58	54

What changed. The untrained model never produces the required JSON (0% everywhere) because it has never seen the format. After fine-tuning on 1,500 teacher replies it matches the teacher on this sample (58 vs 57 full-frame). The 1-point

difference is within noise on 100 examples and I do not claim the student is better. A plausible reason it is not worse is that training only used replies that passed cleaning rules (a) to (d), whereas the teacher's own score includes its unfiltered mistakes (A). The final loss of 0.016 shows the student largely memorised its training set, so the validation numbers are the fair measure. v2 (one more epoch) is slightly lower, consistent with more over-fitting.

Sequence-level vs logit-based distillation. Sequence-level distillation (Kim and Rush, 2016) trains the student with ordinary cross-entropy on the text the teacher generated; only the teacher's output string is needed. Logit-based distillation trains the student to match the teacher's per-token probability distribution (for example a KL loss over the vocabulary), which carries information about the alternatives the teacher considered. It is not possible here because I only stored the generated text from Ollama, and in any case phi3.5 and Pythia use different tokenizers and vocabularies, so their output distributions do not line up token for token. **Limitation:** the student can only be as good as the teacher's text, and it inherits its errors; the teacher scored 57% full-frame, so many of the labels it learned from are wrong, and the student has no access to the teacher's uncertainty.

Suitability of Pythia-70M. It is small enough to train on CPU (a few minutes per run) and to serve in under 1 GiB, and for this narrow, fixed-format task it reached 100% valid JSON. It is not a general model and has a short effective context, which is fine for one-line commands.

4. Task 3: the inference service

`service/app.py` (FastAPI, port 8000) has `POST /parse`, `GET /healthz` and `GET /readyz`. At start-up a background thread downloads the five model files from `MODEL_URI` on `S3_ENDPOINT` using credentials from the Secret; `/readyz` returns 503 until loading ends. Configuration comes from MODEL_URI, MODEL_VERSION, S3_ENDPOINT and MAX_NEW_TOKENS (plus FAULT_INJECT). Output is cut at the first newline and checked for exactly four keys, a valid intent and string-or-null slots. Invalid output returns HTTP 500 with `{"error": "invalid_model_output"}`. Responses carry X-Model-Version and X-Served-By (the Pod name).

Check	Measured result
FAULT_INJECT=1	HTTP 500, body {"error": "invalid_model_output"}, headers present
Image	python:3.11-slim, CPU-only torch, pinned requirements, no weights: 1.86 GB (limit 2 GB)
50 sequential requests at 1 CPU, 1 GiB	50 x HTTP 200, OOMKilled=false
Peak memory	356.9 MiB of 1 GiB (docker stats sampled every 1 s, so a shorter spike could be missed) (results/limits_test.txt)

5. Task 4: Kubernetes deployment, scaling and reliability

Setup. In namespace `app`: Deployment (2 replicas, requests 500m CPU / 512 Mi, limits 1 CPU / 1 Gi, liveness on /healthz, readiness on /readyz, maxUnavailable 0, maxSurge 1), Service and a Traefik Ingress sending /parse to the Service. Right after `kubectl apply` the Service had `ENDPOINTS <none>` and the Pods became Ready about 13 s later, so no traffic reaches a Pod that is still loading. The HPA has min 1, max 2, target CPU 70%.

Test (all under the standard Locust load unless noted)	Measured result
Load balancing, 2 Ready replicas	304 successful requests: 153 and 151 (50.3% / 49.7%); the baseline run gave 152 / 149
Rolling update v1 to v2, then rollout undo (first attempt)	about 580 requests, 14 failures (13 x HTTP 502, 1 x HTTP 500), roughly 2.4%: over the 0.5% limit. Cause: old Pods were stopped while the router still sent them traffic
Fix: preStop sleep 10 s and terminationGracePeriodSeconds 40, then rerun	557 requests, 1 failure (HTTP 500), 0.18%, no 502s; responses from v1: 464, v2: 93; after the rollback X-Model-Version is v1 again
Recovery: delete one Pod	replacement Pod Running within 10 s, replica count back to 2; 263 requests, 0 failures
Cold start, all Pods deleted, 3 runs (two Pods each)	Pod creation to Ready: run 1: 7 s, 7 s; run 2: 12 s, 9 s; run 3: 7 s, 7 s. Median of all six: 7 s; median of the three per-run medians (7, 10.5, 7): 7 s. Timestamps have 1 s resolution. Logs show each Pod downloading the five files from s3://models/v1/
HPA 1 to 2 to 1	1 replica idle; at about 15 min CPU rose to 200%/70% and replicas went to 2; after the load ended CPU fell to 0% and replicas returned to 1 about 5 min later (results/hpa_scaleup_watch.txt)

Limitation. The whole cluster is one k3d node on one laptop. This does **not** demonstrate fault tolerance across machines: if the laptop or Docker stops, everything stops. Both replicas also share the same CPUs, so scaling from 1 to 2 Pods adds no real capacity; the experiment shows that the autoscaling mechanism works, not that throughput improves. The SeaweedFS volume is a local-path volume with no replication.

6. Task 5: subnetting and network access control

Subnet	Network / prefix	Usable range	Broadcast	Usable hosts
app (1000)	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1022
public (250)	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Working (usable = 2^(32-prefix) - 2): 1000 needs 2^10 = 1024 addresses, so /22; 250 needs 256, so /24; 100 needs 128, so /25; 20 needs 32, so /27. Allocation is largest first, each block right after the previous one (full working in `network/subnets.md`).

NACL (`network/nacl.json`). Inbound: 100 deny 198.51.100.0/24 (all protocols); 110 allow TCP 80 from anywhere; 120 allow TCP 443 from anywhere; 130 allow TCP 22 from 10.20.5.128/27; 140 allow TCP 1024-65535 from 10.20.0.0/22 (replies from the app subnet). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024-65535 to 0.0.0.0/0 (replies to clients). Everything else is the default deny. Measured: nacl_sim.py gave 6 of 6 correct on the sample packets, and 8 of 8 on eight extra edge-case packets I wrote (results/nacl_check.txt).

Why replies need their own rules. A NACL is stateless: it does not remember that an outgoing request belongs to an incoming reply, so each packet is judged alone and reply traffic on ephemeral ports needs explicit rules in the opposite direction. **Why rule order matters.** Rules are checked from the lowest number and the first match decides. The deny for 198.51.100.0/24 is rule 100, below the allows for ports 80 and 443; if it had a higher number, a packet from that range to port 443 would match the allow first and the block would never apply.

7. Task 6: reproducibility, testing and CI/CD

Everything is in the repository. `make up` builds both images, creates (or starts) the cluster, creates the namespaces and Secrets (S3 keys generated at deploy time), deploys SeaweedFS and imports the images; `scripts/seed.sh` then creates the buckets and uploads the data and models, and `kubectl apply -f k8s/app.yaml -f k8s/hpa.yaml` starts the service (three commands after cloning). `make down` deletes the cluster. There are 6 unit tests (cleaning rules a, c, d, validity, duplicate handling, shard selection) and ruff linting.

The GitHub Actions workflow `ci.yml` chains five stages with `needs`: lint, unit tests, model evaluation (downloads the model from the GitHub Release, predicts on validation.jsonl, runs score.py), quality gate (`score.py --min-validity 95`) and gitleaks. The model tag is a workflow input.

Run	Link	Result
Passing, model v1 (push)	https://github.com/akash2101h/cloud-computing-assignment/actions/runs/42449324700	all five stages green, no leaks detected
Failing, untrained Pythia (tag base, manual)	https://github.com/akash2101h/cloud-computing-assignment/actions/runs/42449897580	lint, tests and evaluation green; quality-gate failed (exit code 1), secret scan skipped

AI use is described in `AI_USE.md`.

8. Task 7: performance, cost and review

Measurement (standard Locust command, 4 users, 2 min, 2 replicas)	Measured
Requests / failures	301 / 0 (error rate 0.00%)
Median (p50) and p95 latency	850 ms and 3100 ms
Throughput	2.53 requests/s
kubectl top pods during the test	parser-...fgmt7: 513m CPU, 544 Mi; parser-...gfc7h: 1001m CPU, 581 Mi
Cold start (from Task 4)	median 7 s

A second identical run gave 304 requests, 0 failures, p50 890 ms, p95 3100 ms, 2.54 requests/s, so the result is stable. The latency is dominated by about one second of CPU generation per request.

Cost with the hypothetical rate card (INR 3.50 per vCPU-hour, INR 0.45 per GiB-hour; each replica is billed for its limits of 1 CPU and 1 GiB):

Step	Calculation	Result
Cost of one replica per hour	1 x 3.50 + 1 x 0.45	INR 3.95
Hourly cost of 2 replicas	2 x 3.95	INR 7.90
Requests served per hour	2.5277 x 3600	9,099.7
Cost per 1,000 requests	7.90 / 9,099.7 x 1000	**INR 0.868**

SLO. Objective: 99% of requests return HTTP 200. Window: one load-test run (301 requests). Error budget: 1% of 301 = 3 failed requests. Used: 0 of 3 (0%).

Review table

Concern	Evidence (measured)	Limitation found	Improvement
Data and storage	3000 labelled, 2831 kept, 104 rejected by rule (d); 3000 unique ids	One SeaweedFS Pod on one local volume; no redundancy	Replicated volumes and backups of datasets and models
Model quality	Student full-frame 58% vs teacher 57%; validity 100%	Final loss 0.016 (memorisation); prompt tuned on 20 validation commands; only 100 validation examples	Separate dev set for prompt tuning; larger test set

Concern	Evidence (measured)	Limitation found	Improvement
Latency and throughput	p50 850 ms, p95 3100 ms, 2.53 req/s	About 1 s CPU generation per request; Pods share one CPU	Fewer new tokens, early stop at EOS, quantisation
Resource utilisation	513m and 1001m CPU, 544 and 581 Mi under load	One Pod hits its 1-core limit; memory use above the 512 Mi request	Raise memory request to about 640 Mi; size CPU from measurements
Reliability and recovery	Rolling update 0.18% failures; recovery 0 failures; cold start 7 s	First rollout had about 2.4% failures until preStop was added; single node	PodDisruptionBudget; multi-node cluster
Security and cost	gitleaks: no leaks; INR 0.868 per 1,000 requests	Same S3 keys in three namespaces; no TLS; Ollama listens on 0.0.0.0	Per-service credentials, TLS, NetworkPolicies, bind Ollama to the cluster interface

9. Assumptions, limitations and honesty notes

• (A) The rate card is hypothetical; the cost is an estimate and no real charge was incurred.

• (A) The hidden test set is unseen. My validation numbers (validity 100%, full-frame 58%) suggest the service will meet the 95% validity and half-of-teacher requirements, but this is not measured on the hidden set.

• (A) I suggest the student's lead over the teacher comes from training only on cleaned replies; I did not run an experiment to prove it.

• The teacher prompt was tuned while looking at the first 20 validation commands, so the teacher's validation score may be slightly optimistic. The prompt was locked before the full labelling run.

• The validation set was never used for training.

• HTTPS from WSL was blocked during the project, so the Pythia-70M files were downloaded in a Windows browser and copied in; they are the same weights as EleutherAI/pythia-70m. The config in the repository names EleutherAI/pythia-70m. The GitHub push and Releases were made from Windows.

• The single-node limitation of Section 5 applies to every reliability result.

• The first rolling update missed the 0.5% target (about 2.4%); the numbers reported as the result come from the rerun after the fix, and the first run's logs are kept in results/rolling_run1_before_fix.log.